Firmware Analysis: Serial Flash Read-out

Giga-tronics 8650 Universal Power Meter (8600A series)

Image: `8600_155.bic` - static reverse-engineering report - 9 Oct 2026, revised 13 Oct 2026

Revision 13 Oct 2026. The load base is **0x6000**, not 0x6010 as stated in the first edition. The handler addresses there are file offsets, not runtime addresses. This edition also corrects the image size and the serial port address, derives the baud rate, and describes how the monitor is entered. All changes are listed in section 8.

1. Summary

Question: does the firmware contain a command to read the contents of flash out over the serial port, given that a serial *upload* procedure is already documented?

Answer: yes. Alongside the normal SCPI instrument interface the firmware carries a low-level serial **service monitor** with a Peek/Poke memory interface. Its `PEH` ("Peek, Hex") command reads bytes from any address in the CPU address space and returns them as hexadecimal text. Flash is memory-mapped, so `PEH` reads it directly. There is no authentication gate on the command.

The monitor runs at power-on: after the self-test it listens for about 1-2 s for the attention byte `*`, then hands over to the instrument application. A Python dumper (`gt8650_dump.py`) was produced. It was validated offline against the supplied image, including a byte-identical full read-back from a simulated device.

2. What the image is

Property	Finding	Evidence
Instrument	Giga-tronics 8650 Universal Power Meter (8600A series)	Banner strings "(C) Giga-tronics, 1999", "Gigatronics 8650 Universal Power Meter", "OK 8600A"
CPU / arch	Motorola MC68340 (CPU32), big-endian	LINK A6 / UNLK A6 / RTS epilogues; MOVEC, MOVES; MBAR write 0xFFFFF101
RTOS	Nucleus PLUS 68340D v1.11.4	"Copyright (c) 1993-1999 ATI - Nucleus PLUS - 68340D Version 1.11.4"
Image size	1,080,718 bytes (0x107D8E)	File length; equals the length field at file 0x414
File format	Raw flash image of 0x6000-0x10DD8D; no file header or trailer	File offset 0 is the vector table (SP 0x2D8FFC, PC 0x9D2A, VBR = 0x6000); file size = length field, so nothing is appended
File hashes	CRC32 7414aad0; SHA-256 6fe06238...44f26cf (full value in README)	Taken over the whole file; identify the file only, not checked by the firmware
Internal checksum	Field at file 0x418 (flash 0x6418) in the boot header	Algorithm and covered range not analysed
Load mapping	runtime = file_offset + 0x6000	VBR is set to 0x6000; cross-checked 5 ways (see sect. 5)
Flash	CS0 0x000000-0x0FFFFF, CS1 0x100000-0x1FFFFF	Chip-select table at file 0x301A
RAM	CS2 0x200000-0x2FFFFF (initial SP 0x2D8FFC, reset code sets 0x2FFFF0)	Chip-select table; RAM test 0x200000-0x2FFFFF
Serial UART	MC68340 on-chip serial module, channel A at 0xFFFFF700	Absolute-short `$F711.w` / `$F713.w` sign-extend to 0xFFFFF711 (SRA) / 0xFFFFF713 (RBA/TBA)
Monitor line settings	9600 baud, 8N1	UART init at file 0x1774: CSR 0xBB, ACR 0x80, MR1 0x93, MR2 0x07

3. Method

- Classified the image and extracted ASCII / big-endian strings; swept for memory / flash / dump / upload vocabulary.
- Recognised the m68k/CPU32 code and mapped the SCPI command parse-tree (no raw-flash read command there).
- Disassembled with Capstone (M68K, big-endian) to locate and decode the serial service monitor.
- Resolved PC-relative string cross-references (base-independent) to label handlers.
- Pinned the absolute load base from the VBR setup, the reset vector, the exception-vector stub table, the exception vectors and the self-relocation code.
- Followed the boot path from the image header into the monitor, and decoded the chip-select and UART initialisation.
- Decoded the command dispatch and the Peek/Poke handlers; characterised the on-wire framing end to end.
- Wrote a dump tool and validated it against a simulated monitor (power-on window, active monitor, full read-back).

4. The serial service monitor

Entered by sending the attention byte `*` during the power-on window (section 6). The monitor replies `OK 8600A`, or `ERR 8600A` if the power-on self-test flagged a fault. Reads still work after `ERR 8600A`.

It then stays in a loop reading CR- or LF-terminated lines (ESC discards the line, backspace edits, `*` is ignored). Each line is upper-cased and dispatched on its first letter through a 26-entry jump table at file offset 0x12B0. The `P` command branches further into Peek (`PE*`) and Poke (`PO*`) families.

Addressing note: all handler locations below are **file offsets** in `8600_155.bic`. The monitor executes from a RAM copy, so add 0x2FBC00 for the RAM address (file - 0x400 + 0x2FC000), or 0x6000 for the flash address.

4.1 Memory read (Peek) commands

Command	Handler (file offset)	Action
PEH <n><addr>	0x1A88	Read n bytes from addr, return as uppercase hex (the flash-dump command)
PEA <n><addr>	0x1B48	Read bytes from addr, return as a null-terminated ASCII string
PEU / PES / PED <sz><addr>	0x1BE4 / 0x1CD0 / 0x1DBC	Read one value, print as decimal
POH / POA / POU / POS / POD	0x1E60 / 0x1F24 / 0x1FC4 / 0x2090 / ...	Poke - WRITE values into memory (used by the upload/program path)

<n> is a single hex digit (1-F); `0` selects the default of 0x28 = 40 bytes. `<addr>` is a 32-bit hex address; spaces are skipped. The command is dispatched directly - no unlock or password is required.

4.2 On-wire exchange for a hex read

```
host -> *                     (attention, during power-on window)
dev  -> OK 8600A<CR><LF>      (banner)
host -> PEH06000<CR>          count '0' = 40 bytes, addr 0x6000
dev  -> 002D8FFC00009D2A...<CR><LF>   80 hex chars = 40 bytes
```

Nibble encoding is uppercase (encoder at file 0x2328). The reply is built in the line buffer and sent with CR+LF appended by the put-string routine at file 0x18D4. Command echo is off by default. `E1` turns it on and `E0` turns it off (handler at file 0x136C).

5. Load base and memory map

The image is linked for **0x6000**. Evidence:

- The application start-up code executes `MOVE.L #$6000,D0 / MOVEC D0,VBR` (file 0x303A). The vector table at file offset 0 is therefore the live table at 0x6000.
- The reset vector 0x9D2A maps to file 0x3D2A, which is `MOVEA.L #$2FFFF0,A7 / MOVEA.L #$2C4768,A5 / JSR $8FA6`. 0x8FA6 (file 0x2FA6) is `MOVE.W #$2700,SR / MOVEC ... DFC`, the classic CPU32 cold-start sequence. With base 0x6010 both land in the middle of instructions.
- The 254 BSR.W exception stubs (stride 4) start at file 0x3108 = runtime 0x9108. The common handler they branch to computes the vector number with `SUB.L #$9108,D7`.
- The bus-, address-error, illegal and zero-divide vectors (0x95F6, 0x9616, 0x9656, 0x9636) all land on identical handler prologues (`ORI.W #$700,SR / MOVEM.L D0-D7/A0-A6,-(A7)`).
- The image contains self-relocation code linked at 0x6000 (`MOVEA.L #$6414,A0 / SUBA.L #$6000,A0 ...`).

Consequence for dumping: the bytes of this image appear on the bus at **0x00006000 ... 0x0010DD8D**.

Range	Contents
0x000000-0x003FFF	Boot block (not part of the .bic; contents unknown)
0x004000-0x005FFF	Data block with length + checksum, checked at boot (probably calibration/configuration; not part of the .bic)
0x006000-0x0063FF	Exception vector table (VBR = 0x6000)
0x006400-0x10DD8D	Boot header from 0x6400 (SP, jump to boot entry (0x6408), image length 0x107D8E (0x6414), checksum (0x6418), application entry (0x641C), monitor entry (0x6420)), followed by application code and data
0x200000-0x2FFFFF	RAM; the monitor runs from a copy at 0x2FC000
0x800000-	Peripherals (CS3)
0xFFFFF000-	MC68340 on-chip modules (MBAR)

The boot header is part of the firmware, not a container header of the .bic file. A dump from 0x000000 to 0x1FFFFF captures both flash chip-selects. That includes the boot block and the data block, which are not in the .bic.

6. Entering the monitor

The boot path (addresses from 0x6000 upward lie in the image; subtract 0x6000 for the file offset):

1. Boot entry 0x6408 jumps to 0x6854: SIM/chip-select initialisation, then a RAM pattern test over 0x200000-0x2FFFFF.
2. If the RAM test passes, the data block at 0x4000 is checked (length word at 0x4000, at most 0x2000; checksum). The result is stored as a fault code: 0x10 = RAM fault, 0x04 = data-block fault.
3. 16 KB of monitor code is copied to RAM at 0x2FC000 and entered at 0x71C4 (RAM 0x2FCDC4).
4. The monitor initialises channel A to 9600 8N1 and polls for a received byte with a 0xFFFFF-iteration timeout (estimated 1-2 s).
5. If * arrives, it prints the banner and enters the command loop. On a timeout with no fault, it starts the instrument application (0x641C). With a fault it keeps waiting for *.

The practical procedure is to send * repeatedly while switching the instrument on. Bytes sent before the UART initialisation are discarded, because that initialisation resets the receiver. Once the monitor is in its command loop it ignores further * characters, so a host can tell an active monitor apart by sending a `PEH` command.

7. Deliverable: gt8650_dump.py

A self-contained Python 3 tool (pyserial) that drives the monitor and reassembles a binary:

- Detects an already active monitor via a test `PEH`; otherwise sends `*` repeatedly (up to `--wait`, default 60 s) until the `8600A` banner arrives, so the user can start the script and then power the unit on.
- Reads in 40-byte blocks via `PEH0<addr>`, parses the hex reply, retries on short or garbled lines, and shows progress with an ETA.
- Supports arbitrary `--start` / `--length`. The default start is 0x6000, and `--base` overrides the mapping for other firmware versions.
- `--verify` compares the part of the dump that overlaps a reference .bic placed at the base address.
- `--probe` reads 0x0, 0x4000 and 0x6000 and confirms the mapping against the reference.

Typical use:

```
# start the script, then switch the instrument on
python3 gt8650_dump.py -p /dev/ttyUSB0 --probe --verify 8600_155.bic

# re-dump this image's range and verify it matches
python3 gt8650_dump.py -p /dev/ttyUSB0 -o firmware.bin --verify 8600_155.bic

# whole flash incl. boot block and data block
python3 gt8650_dump.py -p /dev/ttyUSB0 --start 0 --length 0x200000 \
        -o flash_full.bin --verify 8600_155.bic
```

At 9600 baud one 40-byte block costs about 92 characters on the wire, so throughput is at most about 400 B/s. Even at that rate the image takes about 45 min and the full 2 MB flash about 85 min.

8. Changes from the first edition (9 Oct 2026)

First edition	Corrected
runtime = file_offset + 0x6010	runtime = file_offset + 0x6000 (VBR, reset vector, stub table, exception vectors, self-relocation code)
Image size 0x107ECE; bus range 0x6010-0x10DEDE	Image size 0x107D8E; bus range 0x6000-0x10DD8D
Jump table "at runtime 0x12B0"; handlers 0x1A88 etc.	These are file offsets; RAM copy at +0x2FBC00, flash at +0x6000
DUART at 0x00F700	MC68340 on-chip serial module at 0xFFFFF700
Baud rate cannot be computed	9600 8N1 from the UART initialisation (standard 3.6864 MHz clock assumed)
Monitor loop has no locatable caller	Boot path 0x6408 -> 0x6854 -> 0x6C08 -> RAM copy; * window at power-on
PEH0 6010 / dumper default 0x6010	PEH0 6000 / dumper default 0x6000

9. Open items and caveats

- **Findings are static.** The PEH read path, the absence of an auth gate, the load base and the UART settings are unambiguous in the code. The dumper has been validated only against a simulated monitor and should be confirmed on a live unit with `--probe`.
- **Power-on window.** The 1-2 s figure is an estimate from the poll loop; the dumper therefore sends * continuously.
- **Checksums.** The checksum in the boot header (0x6418) and the one in the data block (0x4000) have not been analysed: neither the algorithm nor the covered range is known.
- **Boot block.** The code at 0x0 is not in this image. That it loads the same monitor layout is inferred from the entry-address calculation (0x71C4 - 0x6400 + 0x2FC000), not proven.
- **Baud clock.** 9600 baud assumes the standard 3.6864 MHz serial clock; if no banner appears, try 19200 and 115200.